

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Viện Trí Tuệ Nhân Tạo



PHƯƠNG PHÁP LUẬN LẬP TRÌNH

BÁO CÁO
LEVEL_2

Sinh viên thực hiện

Đỗ Minh Trí

Mã sinh viên: 25023059

Hà Nội, năm học 2025 – 2026

MỤC LỤC

I. Tổng quan (Overview) 1

II. Chi tiết cài đặt (Implementation Details) 1

III. Kết quả thực nghiệm (Experiment Results) 9

IV. Nhận xét và tổng kết bài làm 12

V. Nguồn tham khảo (References). 15

VI. Ghi chú sử dụng AI 15

I. Tổng quan (Overview)

Thông tin cá nhân:

- **Họ và tên:** Đỗ Minh Trí
- **Mã sinh viên:** 25023059

Tổng quan thiết kế luồng thực thi:

Chương trình vẫn giữ lối tuần tự của starter code: `parseArgs()` → `init` → `Engine::init()` → `startGame()` → `playGame()` → `endGame()` → `close()`. Điểm được mở rộng nằm ở tầng hạ tầng và `GUI.main.cpp` chọn implementation cụ thể cho `I_Renderer` và `I_Interaction`; nếu chạy Terminal hoặc judge mode thì dùng `TerminalRenderer` / `TerminalInteraction`, nếu chạy GUI thì dùng `SDLRenderer` / `SDLInteraction`, đồng thời gắn thêm module `Audio`. Ở GUI, trước mỗi ván chơi, `main.cpp` gọi `sdlInter->handleMainMenu()` để hiển thị menu chính gồm `PLAY`, `SETTINGS`, `ABOUT`, `QUIT`. Sau khi chọn `PLAY`, Engine mới chạy vòng đời game. Điều này giúp GUI có trải nghiệm giống một ứng dụng hoàn chỉnh hơn, không chỉ là một cửa sổ bàn cờ.

Giao diện đồ họa (GUI):

- **Hình thức hiển thị:** GUI dùng **SDL2**, **SDL_ttf**, **SDL_image** và **SDL_mixer**. Bàn cờ và quân X/O được vẽ bằng primitive drawing, còn nền, logo và một phần trải nghiệm menu dùng texture ảnh.
- **Kiểu tương tác hỗ trợ:** Bàn phím dùng để nhập số, xác nhận bằng `Enter`, điều hướng menu bằng phím mũi tên, thoát bằng `Escape`; chuột dùng để chọn menu, thao tác settings và click trực tiếp lên ô bàn cờ.

II. Chi tiết cài đặt (Implementation Details)

Tư duy thiết kế: Trình bày chi tiết cách xây dựng luồng thực thi của Engine. Có thay đổi gì so với luồng ban đầu? Tại sao lại lựa chọn thiết kế này? Thiết kế này có ưu điểm và nhược điểm gì?

Luồng lõi của Engine vẫn là luồng tuần tự gồm năm bước: `init()`, `startGame()`, `playGame()`, `endGame()`, `close()`. Trong `startGame()`, Engine lần lượt hỏi người chơi kích thước bàn cờ, điều kiện thắng, chế độ chơi và level bot. Trong `playGame()`,

Engine lập qua từng lượt: render bàn cờ, lấy nước đi từ human hoặc bot, cập nhật Logic, kiểm tra thắng/hòa rồi chuyển lượt. Trong `endGame()`, Engine lấy `WinLine` nếu có, hiển thị kết quả và ghi log.

Thay đổi lớn so với luồng ban đầu nằm ở `main.cpp`: GUI được bọc trong `while(keepRunning)` để có thể quay lại menu và chơi nhiều ván trong một lần mở chương trình. Menu chính được xử lý bởi `SDLInteraction::handleMainMenu()`, còn Engine chỉ chạy khi người chơi chọn `PLAY`. Khi người chơi bấm `QUIT` hoặc đóng cửa sổ, `QuitException` được ném lên để `main.cpp` bắt và dọn tài nguyên.

Lý do chọn thiết kế này là để giữ Engine độc lập với giao diện. Engine không cần biết đang chạy Terminal hay SDL; nó chỉ gọi các hàm của `I_Renderer` và `I_Interaction`. Nhờ vậy, cùng một logic game có thể chạy ở Terminal, judge mode hoặc GUI.

Ưu điểm: luồng tuần tự dễ đọc, dễ debug, phù hợp với starter code và dễ ánh xạ sang các bước của game. Interface giúp giảm phụ thuộc giữa Engine và UI, đồng thời thuận tiện khi thêm renderer hoặc interaction mới.

Nhược điểm: SDL vốn phù hợp với event-driven loop, trong khi Engine hiện tại hoạt động theo kiểu đồng bộ. Vì vậy, các hàm input của `SDLInteraction` phải tự viết vòng lặp `SDL_PollEvent` / `SDL_WaitEvent` để chờ người dùng. Ngoài ra, do menu GUI cần renderer trước khi Engine chạy, bản hiện tại có sự chồng lấn trách nhiệm khởi tạo giữa `main.cpp` và `Engine::init()`; về lâu dài nên tách một `Application` hoặc `InfraManager` để quản lý init/close sạch hơn.

1. Lập trình hướng đối tượng: Trò chơi của bạn có thể hiện đầy đủ 4 nguyên lý của OOP không? Bên cạnh những gì starter code đã cung cấp, bạn có cài đặt thêm hoặc sáng tạo thay thế để làm rõ nguyên lý OOP? Chỉ ra minh chứng cho từng nguyên lý.

Đóng gói (Encapsulation):

Project thể hiện đóng gói rõ ở các lớp chính. Engine giữ `config`, `iRenderer`, `iInteraction` và `gameSetup` ở phạm vi `private`; bên ngoài chỉ có thể điều khiển Engine qua các hàm `public` như `init()`, `startGame()`, `playGame()`, `endGame()` và `close()`. `SDLRenderer` cũng đóng gói các con trỏ tài nguyên SDL như `SDL_Window*`, `SDL_Renderer*`, `SDL_Texture*`, `font`, danh sách sao, `particle` và trạng thái menu. Các chi tiết như `drawBg()`, `drawStarField()`, `spawnParticles()` được giữ `private`, không để Engine gọi trực tiếp.

```

class Engine
{
private:
    const RunConfig *config;
    I_Renderer *iRenderer;
    I_Interaction *iInteraction;
    GameSetup gameSetup;
    bool sanity_check();
public:
    Engine(const RunConfig *_config, I_Renderer *_iRenderer, I_Interaction *_iInteraction);
    ~Engine();
    void init();
    void startGame();
    GameResult playGame();
    void endGame(const GameResult &gameResult);
    void close();
};

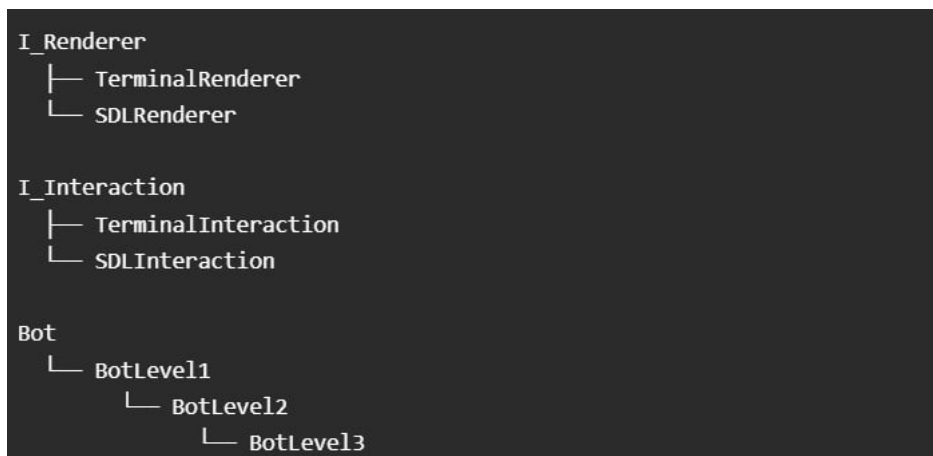
```

Hình 1. Encapsulation trong Engine

Kế thừa (Inheritance):

Project có ba nhóm kế thừa chính:

- I_Renderer là base class cho TerminalRenderer và SDLRenderer.
- I_Interaction là base class cho TerminalInteraction và SDLInteraction.
- Bot là abstract base class cho cây bot: BotLevel1 → BotLevel2 → BotLevel3. BotLevel2 kế thừa random pick của BotLevel1; BotLevel3 kế thừa heuristic cơ bản của BotLevel2 và bổ sung scoring nâng cao.



Hình 2. Sơ đồ kế thừa trong project

Trừu tượng (Abstraction):

I_Renderer và I_Interaction định nghĩa những hành vi mà Engine cần, thay vì ép Engine biết chi tiết Terminal hoặc SDL. Ví dụ, Engine chỉ gọi `iRenderer->displayBoard(...)` hoặc `iInteraction->getPlayerMove(...)`. Cách lấy input bằng `std::cin`, bằng file input, hay bằng mouse event đều bị ẩn phía sau interface. Tương tự, Bot định nghĩa

abstract method `getMove()`, giúp Engine không cần biết thuật toán cụ thể của từng bot.

```
class I_Renderer {
public:
    I_Renderer();
    virtual ~I_Renderer();
    virtual void init(const RunConfig& config) = 0;
    virtual void clearScreen() = 0;
    virtual void showSelectMenu(SelectType selectType, int context = NO_CONTEXT) = 0;
    virtual void showInvalidSelect(SelectType selectType, int context = NO_CONTEXT) = 0;
    virtual void showValidSelect(SelectType selectType, int context = NO_CONTEXT) = 0;
    virtual void displayBoard(const char board[][BOARD_N_MAX], const int size) = 0;
    virtual void showMove(const int row, const int col) = 0;
    virtual void showInvalidMove() = 0;
    virtual void showPlayer(const int player, const bool is_bot) = 0;
    virtual void showResult(const int winner, const bool is_bot, const winLine* winLine = nullptr) = 0;
    virtual void printResult(const GameResult& gameResult) = 0;
    virtual void close() = 0;
};
```

Hình 3. Abstraction trong iRenderer

Đa hình (Polymorphism):

Đa hình xuất hiện khi Engine dùng con trỏ base để gọi hành vi cụ thể tại runtime. Cùng một lời gọi `iRenderer->showResult(...)` có thể in kết quả ra terminal hoặc vẽ màn hình kết quả bằng SDL. Cùng một lời gọi `bots[player]->getMove(...)` có thể chạy random pick, heuristic chặn thắng, hoặc hard-level scoring tùy object thực thể do BotFactory tạo ra. Đây là runtime polymorphism qua virtual function.

```
if (config->interactive)
    iRenderer->clearScreen();

if (config->interactive)
    iRenderer->displayBoard(gameSetup.board, gameSetup.size);

if (config->interactive)
    iRenderer->showPlayer(player, is_bot[player]);
pii point = measureExecutionTime(
    std::format("bot#{}->getMove()", player),
    [&]()
    {
        return bots[player]->getMove(
            gameSetup.board,
            gameSetup.size,
            gameSetup.goal);
    },
    TIME_ENABLED);
if (config->interactive)
{
    iRenderer->clearScreen();
    iRenderer->displayBoard(gameSetup.board, gameSetup.size);

    char symbol = (gameResult.winner == 0) ? 'X' : 'O';

    auto winLine = Logic::getWinLine(
        gameSetup.board,
        gameSetup.size,
        symbol,
        gameSetup.goal);

    iRenderer->showResult(
        gameResult.winner,
        gameResult.isBot,
        winLine ? &(*winLine) : nullptr);
}
```

Hình 4. Đa hình trong Engine

2. Thiết kế cân bằng hai giao diện: Thiết kế của bạn có cân bằng giữa việc xử lý giao diện Terminal và GUI không? Nếu có, đâu là thứ cốt lõi giúp cho thiết kế của bạn hài hoà? Nếu không, đâu là vấn đề khó khăn trong quá trình thiết kế?

Thiết kế cân bằng ở phần lõi: Engine, Logic và Bot không phụ thuộc trực tiếp vào Terminal hay SDL. Sự hài hòa đến từ hai interface `I_Renderer` và `I_Interaction`. Terminal và SDL cùng implement một bộ hàm: `showSelectMenu()`, `displayBoard()`, `showResult()`, `selectSize()`, `selectGoal()`, `getPlayerMove()`, v.v. Nhờ vậy, Engine có thể chạy cùng một vòng đời cho cả hai giao diện.

Tuy nhiên, sự cân bằng chưa hoàn toàn tuyệt đối. GUI hiện có nhiều chức năng riêng hơn Terminal: main menu, settings, about, ảnh nền, logo, OST, volume, animation và particle effect. Những chức năng này không nằm trong interface chung mà được gọi trực tiếp qua `SDLInteraction` trong `main.cpp`. Đây là lựa chọn thực dụng: nếu ép toàn bộ menu GUI vào interface chung thì Terminal sẽ phải implement nhiều hàm không cần thiết. Ngược lại, nếu để GUI tự do hoàn toàn thì Engine dễ bị phụ thuộc vào SDL. Bản hiện tại chọn điểm giữa: lõi game dùng interface chung, còn trải nghiệm ngoài game của GUI được xử lý riêng ở tầng `main.cpp/SDLInteraction`.

Khó khăn lớn nhất là khác biệt mô hình input. Terminal có thể block bằng `cin`, còn SDL cần liên tục poll event để cửa sổ không bị đứng. Vì vậy, `SDLInteraction` phải mô phỏng input đồng bộ bằng các vòng lặp event.

3. Đánh giá mức độ tuân thủ SOLID: Thiết kế của bạn có tuân thủ các nguyên tắc SOLID không? Chỉ rõ từng nguyên tắc đã tuân thủ với minh chứng cụ thể. Đối với các nguyên tắc chưa tuân thủ hoàn toàn, chỉ ra phần vi phạm và diễn giải.

- **S – Single Responsibility Principle (SRP): Đạt khá tốt.** Logic xử lý luật chơi; Engine điều phối vòng đời game; `TerminalRenderer/SDLRenderer` xử lý hiển thị; `TerminalInteraction/SDLInteraction` xử lý input; Audio xử lý âm thanh; `BotFactory` tạo bot. Điểm chưa thật sạch là `SDLRenderer` đang vừa render game, vừa render main menu, settings và about screen. Nếu project lớn hơn, nên tách thành các screen hoặc scene riêng.
- **O – Open/Closed Principle (OCP): Đạt cơ bản.** Có thể thêm renderer mới bằng cách kế thừa `I_Renderer`, thêm interaction mới bằng cách kế thừa `I_Interaction`, thêm bot mới bằng cách kế thừa `Bot`. Tuy nhiên, `BotFactory::createBot()` vẫn phải sửa switch-case khi thêm level bot mới, nên OCP chưa tuyệt đối.
- **L – Liskov Substitution Principle (LSP): Đạt.** `TerminalRenderer` và `SDLRenderer`

đều có thể được dùng qua con trỏ `I_Renderer*`; `TerminalInteraction` và `SDLInteraction` đều có thể được dùng qua `I_Interaction*`. Các bot level cũng trả về cùng kiểu dữ liệu `pII` và tuân thủ kỳ vọng “trả về một nước đi hợp lệ nếu còn ô trống”.

- **I – Interface Segregation Principle (ISP): Đạt tương đối.** `Renderer` và `Interaction` đã được tách riêng. Tuy vậy, `I_Renderer` vẫn chứa `printResult()` dành cho judge mode; GUI phải implement dù hầu như không dùng trong flow bình thường. Ngoài ra, các hàm menu GUI riêng không đưa vào interface chung, điều này tránh làm `Terminal` bị phình interface nhưng cũng khiến GUI có một nhánh xử lý riêng.
- **D – Dependency Inversion Principle (DIP): Đạt tốt ở lõi game.** Engine phụ thuộc vào abstraction `I_Renderer/I_Interaction`, không phụ thuộc trực tiếp vào `SDLRenderer` hay `TerminalRenderer`. Implementation cụ thể được inject từ `main.cpp`. Module bot cũng được tạo qua `BotFactory` và dùng qua con trỏ `Bot`.

4. Sử dụng mẫu thiết kế: Chương trình của bạn có sử dụng thêm bất kỳ mẫu thiết kế nào không? Nếu có, trình bày chi tiết mẫu thiết kế và minh chứng cài đặt.

Chương trình có sử dụng một số mẫu thiết kế đã xuất hiện trong bài giảng về Design Pattern, nổi bật nhất là **Factory** và **Strategy**.

Factory Pattern:

Project sử dụng lớp `BotFactory` để gom logic khởi tạo bot về một nơi duy nhất. Thay vì để Engine tự kiểm tra level rồi trực tiếp tạo `BotLevel1`, `BotLevel2` hoặc `BotLevel3`, chương trình gọi hàm `BotFactory::createBot()`.

```
Bot* BotFactory::createBot(BotLevel level, const char& symbol) {
    switch (level) {
        case BotLevel::EASY:    return new BotLevel1(level, symbol);
        case BotLevel::MEDIUM: return new BotLevel2(level, symbol);
        case BotLevel::HARD:    return new BotLevel3(level, symbol);
        default:                return nullptr;
    }
}
```

Hình 5. Hàm BotFactory

Trong `bot_factory.cpp`, hàm `createBot()` nhận vào `BotLevel` và trả về đúng object bot tương ứng. Cách cài đặt này là một dạng Factory đơn giản: client chỉ yêu cầu tạo bot theo level, còn việc chọn class cụ thể được chuyển cho `BotFactory`.

Strategy Pattern:

Hệ thống bot cũng thể hiện tư tưởng của **Strategy Pattern**. Lớp cha Bot định nghĩa interface chung là hàm `getMove()`, còn mỗi lớp bot con cài đặt một chiến lược chọn nước đi khác nhau.

Cụ thể:

- BotLevel1 chọn nước đi ngẫu nhiên bằng `random_pick()`.
- BotLevel2 dùng heuristic đơn giản: tìm nước thắng, chặn đối thủ, ưu tiên trung tâm.
- BotLevel3 dùng chiến lược khó hơn: kiểm tra thắng/chặn ngay, sau đó chấm điểm ô theo hướng tấn công và phòng thủ.

Trong Engine, bot được dùng qua con trỏ lớp cha Bot. Engine chỉ gọi:

```
bots[player] -> getMove(gameSetup.board, gameSetup.size, gameSetup.goal)
```

Lời gọi này sẽ chạy đúng thuật toán của object thực tế tại runtime. Vì vậy, có thể thay đổi chiến lược bot mà không cần sửa game loop chính của Engine.

Giao diện đồ họa – Hiển thị: Trình bày về hình thức hiển thị trên giao diện GUI. Tập trung vào triết lý/cơ sở cài đặt. Lý do lựa chọn hình thức này? Hiệu năng hiển thị đồ họa? (Có thể dùng kết quả log để tính thời gian render hình ảnh, nếu làm FPS có thể tính trung bình số frame, thời gian xử lý một frame, ...).

SDLRenderer là trung tâm hiển thị của GUI. Renderer khởi tạo `SDL_Window`, `SDL_Renderer`, bật blend mode, load font `PressStart2P-Regular.ttf` ở nhiều kích thước, load ảnh qua `SDL_image` và render âm thanh gián tiếp qua con trỏ `Audio`.

- **Nền và menu:** main menu dùng star field động kết hợp ảnh nền bán trong suốt. Settings screen dùng gradient, preview background, volume bar và thông tin OST hiện tại.
- **Ảnh và texture:** `loadBg()` dùng `IMG_Load()` để load `bg1.jpg` đến `bg5.jpg`; `logo.png` được convert thành `SDL_Texture` để vẽ ở menu chính.
- **Bàn cờ:** lưới bàn cờ được vẽ bằng `SDL_RenderDrawLine`; ô nền dùng `SDL_RenderFillRect`; quân X/O được vẽ bằng hình học cơ bản.
- **Animation:** `animatePlacePiece()` tạo hiệu ứng đặt quân; `animateWinLine()` highlight đường thắng; `spawnParticles()` và `drawParticles()` tạo hiệu ứng hạt

khi có kết quả.

- **Âm thanh:** Audio dùng `SDL_mixer` để phát OST lặp vô hạn, phát click theo người chơi và phát hiệu ứng thắng.

Lý do chọn cách render kết hợp primitive và texture là cân bằng giữa đơn giản và thẩm mỹ. Bàn cờ, quân X/O và UI cơ bản vẽ bằng primitive nên dễ scale theo kích thước màn hình. Ảnh nền và logo giúp giao diện sinh động hơn mà không làm logic render bàn cờ phức tạp. Hiệu năng đủ tốt cho game này vì số lượng object nhỏ: 120 sao, một số particle ngắn hạn, vài texture nền và các shape đơn giản. Điểm cần tối ưu nếu mở rộng là cache texture chữ thay vì tạo texture mới mỗi lần gọi `drawTxt()`.

Hiệu năng hiển thị đồ họa:

Hiệu năng GUI khá tốt vì game có quy mô nhỏ: bàn cờ tối đa 12x12, số lượng object ít, phần lớn được vẽ bằng primitive drawing của SDL như `SDL_RenderDrawLine` và `SDL_RenderFillRect`. Ảnh nền và logo được load thành `SDL_Texture` rồi tái sử dụng, không load lại liên tục.

Các hiệu ứng như star field và particle chỉ có số lượng giới hạn, nên không gây nặng đáng kể. Vòng lặp event dùng `SDL_Delay(16)` để giữ tốc độ khoảng 60 FPS và tránh tiêu tốn CPU quá mức. Điểm có thể tối ưu thêm là cache texture chữ thay vì tạo texture mới mỗi lần gọi `drawTxt()`.

Giao diện đồ họa – Tương tác (Interaction): Trình bày cách chương trình xử lý tương tác người dùng. Chương trình hỗ trợ những tương tác nào? Khó khăn lớn nhất trong quá trình cài đặt tương tác?

`SDLInteraction` xử lý input bằng event của SDL. Các tương tác chính:

- **Main menu:** phím mũi tên lên/xuống để đổi selection, `Enter` để chọn; chuột hover/click vào `PLAY`, `SETTINGS`, `ABOUT`, `QUIT`.
- **Settings:** mũi tên trái/phải đổi background hoặc OST tùy trạng thái background; mũi tên lên/xuống chỉnh volume; phím `B` bật/tắt background image; chuột click vào preview background, volume bar hoặc nút back.
- **Nhập cấu hình game:** người chơi nhập số bằng phím số, xóa bằng `Backspace`, xác nhận bằng `Enter`. Các hàm `selectSize()`, `selectGoal()`, `selectGameMode()` và `selectBotLevel()` đều validate input trước khi trả về cho Engine.
- **Đặt quân:** người chơi click vào bàn cờ. Tọa độ pixel được đổi sang `row/col` dựa trên kích thước bàn cờ và vùng render.

- **Thoát:** đóng cửa sổ hoặc bấm Escape trong một số màn hình sẽ ném `QuitException` để thoát flow hiện tại.

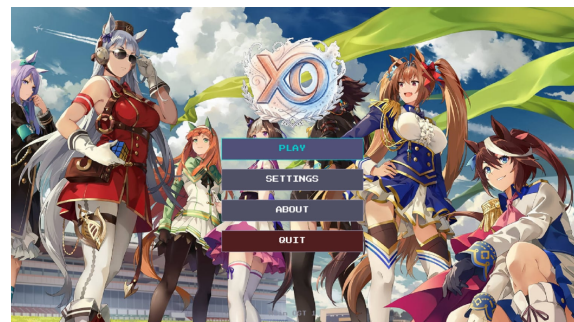
Khó khăn lớn nhất là đồng bộ mô hình event của SDL với Engine tuần tự. Trong Terminal, `cin` tự block. Trong SDL, nếu chỉ chờ input mà không poll event thì cửa sổ sẽ freeze. Vì vậy, các hàm input phải vừa chờ người dùng vừa poll event, delay ngắn 16ms và refresh màn hình khi cần.

III. Kết quả thực nghiệm (Experiment Results)

So sánh giao diện Terminal và GUI:

```
>----- Tic-tac-toe [Console v0.4.25023059] -----<
Enter the board size (NxN, 3 <= N <= 12): 5
Enter the goal (3 - 5, goal <= size): 3
Select Your Game Mode:
(1) PvP | (2) PvE | (3) EVE3
Bot 1 - Select Bot Level:(1) EASY | (2) MEDIUM | (3) HARD3
Bot 2 - Select Bot Level:(1) EASY | (2) MEDIUM | (3) HARD1
```

(a) Terminal – Menu



(b) GUI (SDL) – Menu

```

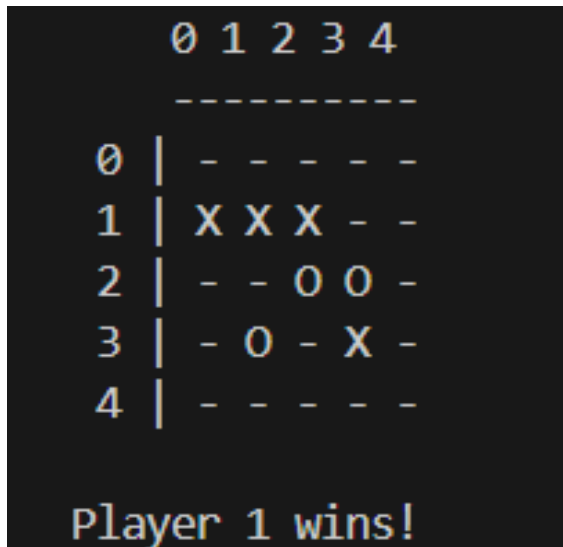
      0 1 2 3 4
      -----
0 | 0 X - X X
1 | - - - - X
2 | - - X 0 X
3 | - - 0 - -
4 | - 0 - - 0

```

(a) Terminal –Bàn cờ



(b) GUI (SDL) – Bàn cờ



(a) Terminal – Kết quả trận đấu



(b) GUI (SDL) – Kết quả trận đấu

Terminal phù hợp cho judge mode, debug và chạy testcase tự động vì output gọn, dễ redirect input từ file. GUI phù hợp cho trải nghiệm người dùng cuối vì có menu, hình nền, logo, animation, âm thanh và thao tác chuột trực tiếp. Hai giao diện dùng cùng Engine, Logic và Bot; khác biệt nằm ở tầng renderer/interaction.

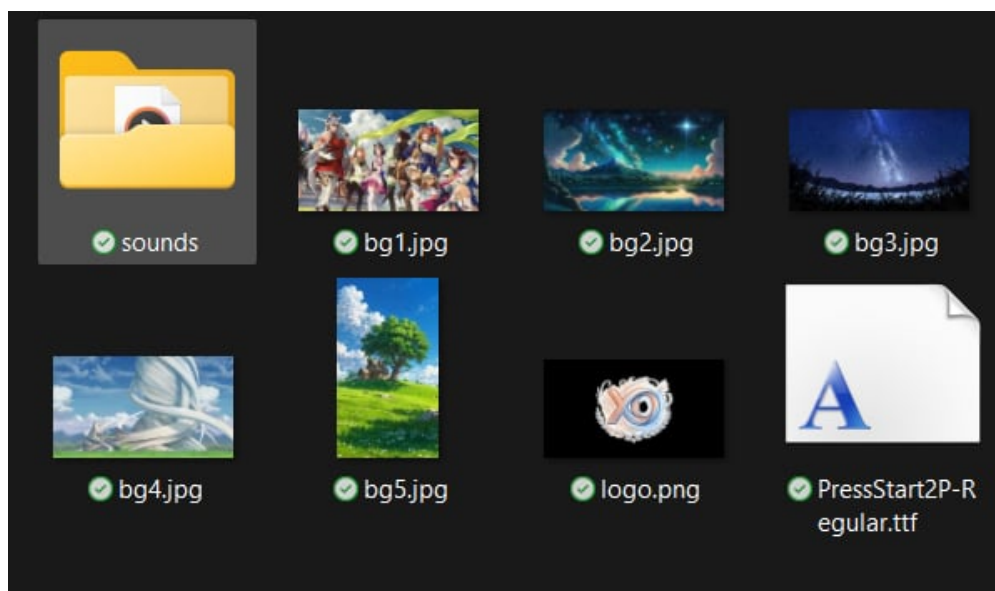
Tài nguyên game (Assets):

Hệ thống tài nguyên ứng dụng trong phiên bản giao diện đồ họa (GUI) được tổ chức phân cấp rõ ràng trong thư mục `assets/`, bao gồm phông chữ, hình ảnh độ phân giải cao và các tệp tin âm thanh kỹ thuật số. Cấu trúc chi tiết của các tài nguyên được mô tả cụ thể dưới đây:

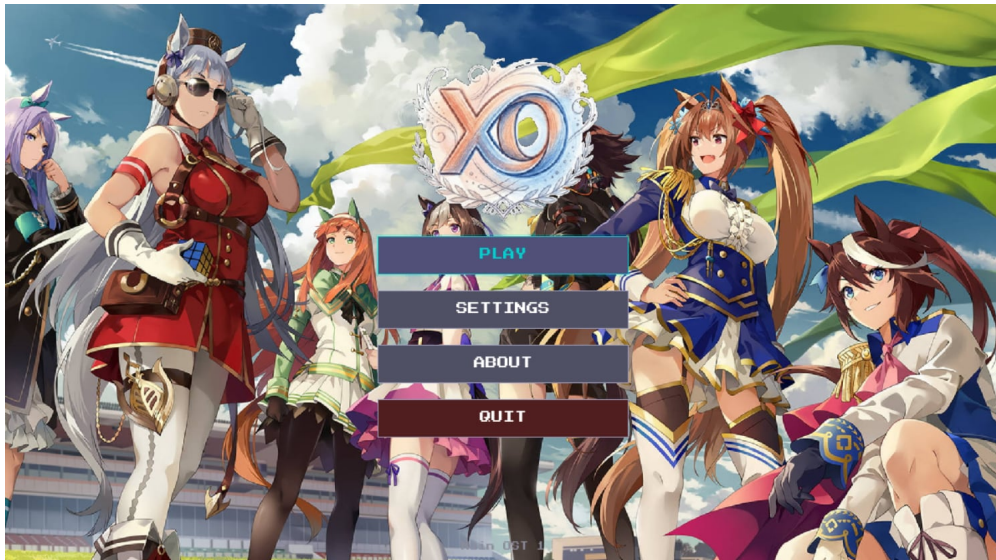
- **Phông chữ (Fonts):** Sử dụng duy nhất phông chữ pixel cổ điển `PressStart2P-Regular.ttf` trên **Google Fonts**. Phông chữ này được nạp vào bộ nhớ thông qua thư viện `SDL_ttf` và phân tách thành 4 biến con trỏ quản lý các kích cỡ khác nhau phục vụ cho từng thành phần giao diện:
 - `fSmall`: Dùng cho các dòng text phụ, thông tin bản nhạc OST đang phát dưới đáy màn hình.
 - `fMedium`: Dùng để hiển thị văn bản trên các nút bấm (`PLAY`, `SETTINGS`,...).
 - `fLarge`: Dùng cho các ký tự đại diện `X` và `O` trên các ô cờ.
 - `fXLarge`: Dùng cho các tiêu đề lớn hoặc thông báo kết quả trận đấu.
- **Hình ảnh (Images):** Toàn bộ đồ họa được xử lý nạp tự động bằng thư viện mở rộng `SDL_image`.
 - `logo.png`: Hình ảnh biểu tượng XO cách điệu nghệ thuật đặt làm trọng tâm ở

Menu chính được thiết kế với Gemini.

- bg1.jpg đến bg5.jpg: Tập hợp 5 ảnh nền chất lượng cao được thu thập trực tiếp từ nền tảng chia sẻ hình nền nghệ thuật **4K Wallpapers**, hỗ trợ tính năng thay đổi hình nền linh hoạt trong phần cấu hình hệ thống.
- **Hệ thống Âm thanh (Audio):** Được vận hành dựa trên thư viện `SDL_mixer`, chia làm hai phân nhóm riêng biệt:
 - **Nhạc nền (Original Sound Tracks - OST):** Gồm 5 bản nhạc chất lượng định dạng .mp3:
 - * 4 file nhạc (ost1.mp3 – ost4.mp3) là các giai điệu nổi tiếng trích từ trò chơi *Deltarune* do **Toby Fox** sáng tạo.
 - * 1 file nhạc (ost5.mp3) là bản nhạc chủ đề sôi động *Umapyoi Densetsu* từ tựa game *Uma Musume: Pretty Derby*.
 - **Hiệu ứng âm thanh (Sound Effects - SFX):** Gồm các tệp âm thanh đặc trưng định dạng .wav độ trễ thấp được khai thác từ nền tảng **Freesound**, bao gồm 2 hiệu ứng nhấn chuột phân tách riêng biệt cho hai người chơi `click_p1.wav`, `click_p2.wav` và âm thanh ăn mừng chiến thắng `win.wav`.



Hình 9. Folder assets



Hình 10. Menu sử dụng background, logo và nhạc nền

Trích dẫn Log:

```
Tic-tac-toe Game (Version: 0.4.25023059 by baluong.87)
[INFO] - [2026-05-21 23:42:00] Logger initialized!
[INFO] - [2026-05-21 23:42:00] SDL Renderer initialized!
[INFO] - [2026-05-21 23:42:00] Audio initialized
[INFO] - [2026-05-21 23:42:00] SDL Interaction initialized!
[INFO] - [2026-05-21 23:42:00] SDLRenderer initialized
[INFO] - [2026-05-21 23:42:00] SDLInteraction initialized
[INFO] - [2026-05-21 23:42:00] Infra initialized!
[INFO] - [2026-05-21 23:42:01] Engine initializing . . .
[INFO] - [2026-05-21 23:42:01] SDLRenderer initialized
[INFO] - [2026-05-21 23:42:01] SDLInteraction initialized
[INFO] - [2026-05-21 23:42:01] Engine initialized!
[INFO] - [2026-05-21 23:42:01] [Engine] Starting game . . .
[INFO] - [2026-05-21 23:42:08] Board initialized!
[INFO] - [2026-05-21 23:42:08] [Engine] Game started!
[INFO] - [2026-05-21 23:42:08] [Engine] Playing game . . .
[WARN] - [2026-05-21 23:42:24] Quit signal received. Cleaning up...
[INFO] - [2026-05-21 23:42:24] Engine closing . . .
```

IV. Nhận xét và tổng kết bài làm

1. Về phong cách lập trình: Thông qua bài tập này, việc sử dụng phong cách lập trình hướng đối tượng (OOP) có điểm mạnh và điểm yếu gì trong việc thiết kế luồng trò chơi? Cách tiếp cận này đối với một dự án lớn hơn có dễ bảo trì hơn? Theo bạn, có phải dự án nào cũng phù hợp với phong cách lập trình này?

OOP giúp tách rõ trách nhiệm giữa các phần: Engine điều phối, Logic xử lý luật, Renderer hiển thị, Interaction nhận input, Bot quyết định nước đi, Audio xử lý âm thanh. Khi project thêm GUI, ảnh nền, menu, âm thanh và nhiều cấp bot, cách tổ chức này giúp code không bị dồn vào một file duy nhất.

Với dự án lớn hơn, OOP dễ bảo trì hơn vì mỗi module có ranh giới rõ. Sửa giao diện SDL không cần sửa Logic; sửa thuật toán bot không cần sửa renderer. Tuy nhiên, OOP cũng tạo thêm boilerplate: interface, factory, class, file header và source riêng. Với script nhỏ hoặc bài toán xử lý dữ liệu ngắn, lập trình thủ tục có thể đơn giản và hiệu quả hơn. Vì vậy, OOP phù hợp khi dự án có nhiều thành phần có trách nhiệm khác nhau hoặc có khả năng mở rộng lâu dài.

2. Về giao diện người dùng: Khi được tận tay phát triển phần mềm có GUI, bạn thấy những phần nào thú vị và gặp khó khăn nào? So với Terminal Interface thì bạn thích giao diện nào hơn? Tại sao?

Phần thú vị nhất là biến một game console thành ứng dụng có menu, hình nền, logo, âm thanh và animation. Việc dùng `SDL_image` để đổi background, `SDL_mixer` để phát OST/click/win sound và particle system để tạo hiệu ứng thắng giúp chương trình có cảm giác giống game thật hơn.

Khó khăn lớn nhất là xử lý event loop. Terminal rất thẳng: chương trình hỏi và `cin` chờ người dùng nhập. SDL thì khác: nếu không poll event liên tục, cửa sổ sẽ đứng hình. Ngoài ra, việc đổi tọa độ chuột từ pixel sang ô bàn cờ cũng cần tính toán cẩn thận theo board size, padding và cell size.

Nếu chơi game, GUI thú vị hơn vì có tương tác chuột và phản hồi trực quan. Nếu debug hoặc chạy testcase, Terminal tốt hơn vì output rõ, nhanh và dễ tự động hóa.

3. Về tổ chức mã nguồn: Mã nguồn được tách ra nhiều file với chú thích kỹ thuật rõ ràng – điều này có hiệu quả cho việc phát triển lâu dài? So sánh ưu/nhược điểm với cách tập trung vào một file? Khi nào nên lựa chọn cách tổ chức này?

Tách mã nguồn thành nhiều file có hiệu quả rõ khi project đã có nhiều module: `game/`, `game/bot/`, `game/interface/`, `sdl/`, `terminal/`, `utils/`. Cách này giúp tìm

code nhanh hơn, mỗi file có phạm vi nhỏ hơn và dễ làm việc song song.

So với một file duy nhất, nhiều file cần cấu hình build phức tạp hơn và phải quản lý include/header cẩn thận. Một file phù hợp với chương trình nhỏ, demo nhanh hoặc bài tập rất đơn giản. Nhiều file phù hợp khi chương trình có nhiều trách nhiệm độc lập, nhiều người cùng phát triển hoặc cần mở rộng về sau.

4. Về thiết kế chương trình: Bên cạnh việc lập trình mã nguồn (mà AI hiện tại đã làm rất tốt), thiết kế vẫn là thứ quan trọng trước khi cụ thể hoá chương trình. Sau phần làm này, bạn nhìn nhận vai trò của con người và AI thế nào trong một quy trình phát triển phần mềm? Liệu con người có thể bị thay thế? Vấn đề của các công cụ AI hiện tại là gì?

AI hỗ trợ tốt khi cần giải thích thư viện mới, viết skeleton code, gợi ý cách xử lý lỗi hoặc đề xuất phương án refactor. Trong project này, AI có thể giúp hiểu SDL, SDL_ttf, SDL_image, SDL_mixer và cách tổ chức interface.

Tuy nhiên, thiết kế tổng thể vẫn cần con người quyết định. Ví dụ, chọn giữ Engine tuần tự hay chuyển hẳn sang state-based game loop là một quyết định trade-off: giữ tuần tự thì dễ bám starter code, nhưng SDL phải mô phỏng input đồng bộ; state-based thì tự nhiên hơn với GUI nhưng phải refactor mạnh. AI có thể đưa ra lựa chọn, nhưng người làm project cần hiểu bối cảnh, deadline và yêu cầu môn học để chọn phương án hợp lý.

Con người chưa thể bị thay thế hoàn toàn ở vai trò thiết kế, kiểm chứng và chịu trách nhiệm sản phẩm. Vấn đề của AI hiện tại là dễ viết code có vẻ đúng nhưng không nắm hết ràng buộc ngầm của project, dễ tạo code không nhất quán nếu prompt thiếu context và đôi khi bỏ sót lỗi thiết kế dài hạn.

5. Bài học rút ra: Khó khăn lớn nhất mà bạn đã gặp và vượt qua được là gì? Đó là vấn đề thiết kế hay lập trình?

Khó khăn lớn nhất là vấn đề thiết kế: làm sao để cùng một Engine chạy được cả Terminal, judge mode và GUI mà không phải viết ba game loop riêng. Ban đầu, rất dễ để code SDL trộn thẳng vào Engine. Sau khi tách vai trò qua `I_Renderer` và `I_Interaction`, Engine trở nên ổn định hơn và GUI có thể phát triển riêng.

Bài học rút ra là cần xác định boundary của hệ thống trước khi thêm tính năng. Nếu boundary đúng, thêm menu, background, audio hoặc bot mới không làm hỏng lõi game. Nếu boundary sai, mỗi tính năng mới sẽ kéo theo sửa nhiều file không liên quan.

V. Nguồn tham khảo (References)

- Tài liệu lý thuyết của môn Phương pháp luận lập trình.
- Tài liệu SDL:
 - [Lazy Foo's SDL2 Tutorials](#)
 - [SDL2 Wiki Page](#)
 - [SDL2_image Documentation](#)
 - [SDL2_ttf Documentation](#)
 - [SDL2_mixer Documentation](#)
- Playlist [SDL2 Tutorial](#) của kênh Youtube **The Builder**

Một số nguồn có thể không được dùng trực tiếp trong code, nhưng được dùng để tham khảo ý tưởng, hiểu thư viện và giải quyết lỗi khi phát triển.

VI. Ghi chú sử dụng AI

Để hoàn thiện project này, các công cụ AI sau đã được sử dụng:

- **Gemini:** Hỗ trợ giải thích cách dùng *SDL*, *SDL_ttf*, *SDL_image*, *SDL_mixer*, giải thích lỗi *compile/runtime* và hỗ trợ chỉnh nội dung *L^AT_EX*.
- **Claude:** Hỗ trợ mở rộng ý tưởng GUI như *menu*, *settings*, hiệu ứng, âm thanh, *background image* và đề xuất cách *refactor* một số phần tương tác.

Sản phẩm này sử dụng AI ở mức hỗ trợ đáng kể, khoảng 50%. AI giúp việc tiếp cận thư viện mới như *SDL* nhanh hơn bằng cách cung cấp ví dụ code, bộ khung cấu trúc để người dùng dựa vào và điều chỉnh theo mong muốn. Tuy nhiên, người thực hiện vẫn cần đọc lại, chỉnh sửa, kiểm tra và quyết định thiết kế cuối cùng để chương trình phù hợp với yêu cầu bài tập.